

**ENSAM Casablanca**  
École Nationale Supérieure d'Arts et Métiers  
Casablanca, Morocco

---

# Scavenger Hunt

## Specification Document

Gamified QR Code Treasure Hunt Platform

---

**Project:** Scavenger Hunt Platform  
**Version:** 1.0  
**Date:** August 25, 2026  
**Status:** Draft  
**Institution:** ENSAM Casablanca

# Contents

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                            | <b>7</b>  |
| 1.1      | Document Purpose . . . . .                     | 7         |
| 1.2      | Project Background . . . . .                   | 7         |
| 1.3      | Target Audience . . . . .                      | 7         |
| <b>2</b> | <b>Project Overview</b>                        | <b>8</b>  |
| 2.1      | Overview . . . . .                             | 8         |
| 2.2      | Platform Objectives . . . . .                  | 8         |
| 2.3      | Use Case Overview . . . . .                    | 8         |
| <b>3</b> | <b>Functional Requirements</b>                 | <b>10</b> |
| 3.1      | Overview . . . . .                             | 10        |
| 3.2      | Core Use Cases . . . . .                       | 10        |
| 3.2.1    | Player (Student) Capabilities . . . . .        | 10        |
| 3.2.2    | Mentor (Administrator) Capabilities . . . . .  | 10        |
| 3.3      | System State Transitions . . . . .             | 10        |
| <b>4</b> | <b>Non-Functional Requirements</b>             | <b>12</b> |
| 4.1      | Overview . . . . .                             | 12        |
| 4.2      | Performance and Scalability . . . . .          | 12        |
| 4.3      | Availability and Reliability . . . . .         | 12        |
| 4.4      | Usability and Accessibility . . . . .          | 12        |
| <b>5</b> | <b>System Architecture</b>                     | <b>13</b> |
| 5.1      | Architecture Overview . . . . .                | 13        |
| 5.2      | Technology Stack . . . . .                     | 14        |
| 5.2.1    | Frontend Mobile Architecture . . . . .         | 16        |
| 5.2.2    | Frontend Web Dashboard Architecture . . . . .  | 16        |
| 5.2.3    | Backend Application Runtime . . . . .          | 16        |
| 5.2.4    | Persistence and Caching Architecture . . . . . | 17        |
| 5.3      | Component Diagram and Subsystems . . . . .     | 17        |
| 5.3.1    | Authentication Service . . . . .               | 17        |
| 5.3.2    | Game Engine Service . . . . .                  | 17        |
| 5.3.3    | QR Code and Validation Service . . . . .       | 18        |
| 5.3.4    | Notification and Real-Time Service . . . . .   | 18        |
| 5.3.5    | Admin and Mentor Management Service . . . . .  | 18        |
| 5.4      | Communication Protocols . . . . .              | 18        |
| 5.4.1    | RESTful API Specification . . . . .            | 18        |
| 5.4.2    | Real-Time WebSocket Protocol . . . . .         | 19        |
| 5.4.3    | SMS Gateway Protocol . . . . .                 | 19        |

|           |                                                         |           |
|-----------|---------------------------------------------------------|-----------|
| 5.5       | Deployment Architecture . . . . .                       | 19        |
| 5.5.1     | Containerization and Service Isolation . . . . .        | 19        |
| 5.5.2     | Nginx Reverse Proxy and Traffic Routing . . . . .       | 20        |
| 5.5.3     | SSL/TLS Termination and Security . . . . .              | 20        |
| 5.5.4     | Data Persistence and Backup Strategy . . . . .          | 20        |
| <b>6</b>  | <b>Database Design</b>                                  | <b>21</b> |
| 6.1       | Overview . . . . .                                      | 21        |
| 6.2       | Entity-Relationship Model . . . . .                     | 21        |
| 6.3       | Core Tables Summary . . . . .                           | 22        |
| <b>7</b>  | <b>User Interfaces and UX Architecture</b>              | <b>23</b> |
| 7.1       | Overview . . . . .                                      | 23        |
| 7.2       | Student Mobile Application (Frontend) . . . . .         | 23        |
| 7.2.1     | Core Mobile Screens . . . . .                           | 24        |
| 7.3       | Mentor Web Dashboard (Backend Admin) . . . . .          | 24        |
| 7.3.1     | Core Dashboard Modules . . . . .                        | 25        |
| <b>8</b>  | <b>Game Mechanics and Business Rules</b>                | <b>26</b> |
| 8.1       | Overview . . . . .                                      | 26        |
| 8.2       | State Machine Lifecycle . . . . .                       | 26        |
| 8.3       | Scoring and Progression . . . . .                       | 26        |
| <b>9</b>  | <b>API Specification</b>                                | <b>28</b> |
| 9.1       | API Overview . . . . .                                  | 28        |
| 9.1.1     | Architectural Principles . . . . .                      | 28        |
| 9.1.2     | Authentication and Rate Limiting . . . . .              | 28        |
| 9.2       | Core Endpoints Summary . . . . .                        | 28        |
| 9.3       | WebSocket Events . . . . .                              | 28        |
| <b>10</b> | <b>Security Architecture</b>                            | <b>30</b> |
| 10.1      | Security Overview . . . . .                             | 30        |
| 10.2      | Core Security Controls . . . . .                        | 30        |
| 10.2.1    | Authentication and Authorization . . . . .              | 30        |
| 10.2.2    | Anti-Cheat and Validation . . . . .                     | 30        |
| 10.2.3    | Infrastructure Security . . . . .                       | 30        |
| <b>11</b> | <b>Testing and Quality Assurance</b>                    | <b>31</b> |
| 11.1      | Overview . . . . .                                      | 31        |
| 11.2      | Testing Methodologies . . . . .                         | 31        |
| 11.3      | Field Testing Protocols . . . . .                       | 31        |
| <b>12</b> | <b>Deployment and Operations</b>                        | <b>32</b> |
| 12.1      | Overview . . . . .                                      | 32        |
| 12.2      | Infrastructure as Code . . . . .                        | 32        |
| 12.3      | Continuous Integration and Deployment (CI/CD) . . . . . | 32        |
| 12.4      | Observability . . . . .                                 | 32        |
| <b>A</b>  | <b>Appendix</b>                                         | <b>33</b> |
| A.1       | QR Code Sample Format . . . . .                         | 33        |

- A.2 Sample API Request and Response . . . . . 33
  - A.2.1 Send OTP Request . . . . . 33
  - A.2.2 QR Scan Submission . . . . . 34
- A.3 Environment Variables Reference . . . . . 35
- A.4 Acronyms and Abbreviations . . . . . 36

# List of Figures

|     |                                                                     |    |
|-----|---------------------------------------------------------------------|----|
| 2.1 | High-Level Use Case Diagram . . . . .                               | 9  |
| 5.1 | System Architecture Overview . . . . .                              | 13 |
| 6.1 | Entity-Relationship Overview of the Scavenger Hunt Data Model . . . | 21 |
| 7.1 | Conceptual Wireframe of the Student Mobile Interface . . . . .      | 24 |
| 7.2 | Conceptual Layout of the Mentor Web Dashboard . . . . .             | 25 |
| 8.1 | Logical Game Flow and State Progression Diagram . . . . .           | 27 |

# List of Tables

|     |                                                 |    |
|-----|-------------------------------------------------|----|
| 5.1 | System Technology Stack Specification . . . . . | 14 |
| 9.1 | API Endpoints Summary . . . . .                 | 29 |
| A.1 | Environment Variables . . . . .                 | 35 |
| A.2 | Acronyms and Abbreviations . . . . .            | 36 |

# Chapter 1

## Introduction

### 1.1 Document Purpose

This software specification document provides the technical blueprint for the ‘Scavenger Hunt’ platform, detailing its architecture, database schemas, and API definitions.

### 1.2 Project Background

The platform is designed to digitize, manage, and gamify treasure hunt competitions at ENSAM Casablanca. By leveraging QR code technology and mobile connectivity, it transforms the physical campus into an interactive digital arena.

### 1.3 Target Audience

- **Players (Students):** Engage with the mobile application to solve clues and locate targets.
- **Mentors (Administrators):** Operate the web dashboard to oversee the event, track team progress, and enforce rules.
- **Developers:** Utilize this document for architectural reference and API integration.

# Chapter 2

## Project Overview

### 2.1 Overview

**Scavenger Hunt** is a gamified, mobile-first platform designed to transform traditional treasure hunts into a dynamic, digitally-managed experience. The platform leverages QR code technology, real-time scoring, and a timed elimination system to create an engaging event for students, while providing mentors with powerful monitoring tools.

### 2.2 Platform Objectives

- **Digitize Gameplay:** Replace paper clues with mobile QR scanning.
- **Real-Time Monitoring:** Provide administrators with live visibility into team progress and locations.
- **Automated Progression:** Enforce time limits, calculate scores dynamically, and automatically eliminate bottom-ranked teams.

### 2.3 Use Case Overview

The platform supports two primary actor types: **Players** (students) and **Mentors** (administrators). Figure 2.1 provides a high-level use case diagram.

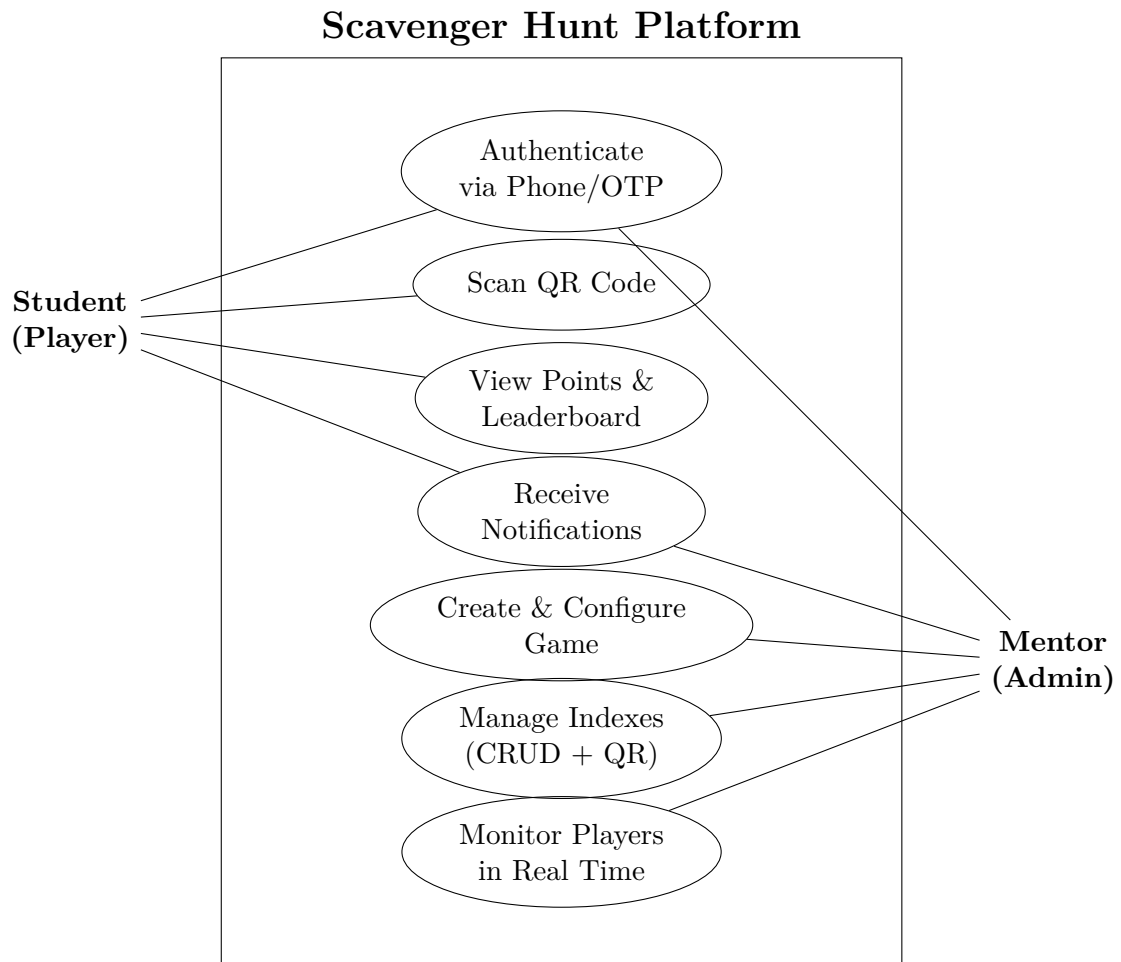


Figure 2.1: High-Level Use Case Diagram

# Chapter 3

## Functional Requirements

### 3.1 Overview

The ‘Scavenger Hunt’ platform provides an interactive, location-based game uniting a cross-platform mobile application for student players with a web-based administrative dashboard for mentors. This chapter outlines the core functional capabilities.

### 3.2 Core Use Cases

#### 3.2.1 Player (Student) Capabilities

- **Authentication:** Passwordless sign-in via SMS OTP.
- **Team Formation:** Create or join a team using a 6-character alphanumeric invite code.
- **QR Scanning:** Access device camera to scan and cryptographically validate physical QR checkpoints.
- **Live Leaderboard:** View real-time team rankings, scores, and elimination status.
- **Progression Tracking:** Monitor countdown timers for the current stage and read next clue hints.

#### 3.2.2 Mentor (Administrator) Capabilities

- **Game Management:** Provision new scavenger hunts, define geographic boundaries, and set elimination parameters (e.g., time limits, score thresholds).
- **QR Generation:** Generate and export cryptographically signed QR codes for physical deployment across campus.
- **Live Telemetry:** Monitor team locations via GPS check-ins on an interactive map.
- **State Override:** Manually advance game states, trigger early eliminations, or award bonus points to specific teams.

### 3.3 System State Transitions

The system enforces strict state machine rules governing the lifecycle of a scavenger hunt round:

1. **PENDING:** Teams are forming; scanning is disabled.
2. **ACTIVE:** The round has started; clues are revealed, and timers begin.
3. **ELIMINATING:** The round timer has expired; the system calculates scores and culls the bottom-performing teams.
4. **FINISHED:** Only the final winning team remains; the event concludes.

# Chapter 4

## Non-Functional Requirements

### 4.1 Overview

Non-functional requirements (NFRs) define the operational quality attributes, constraints, performance benchmarks, and security guarantees that the *Scavenger Hunt* platform must satisfy.

### 4.2 Performance and Scalability

- **Latency:** End-to-end QR code validation and scoring must complete in under 500ms (95th percentile).
- **Concurrency:** The system must support at least 1,000 active concurrent players simulating synchronized scans at round boundaries.

### 4.3 Availability and Reliability

- **Uptime:** The platform targets 99.9% uptime during scheduled event windows.
- **Failover:** In the event of a database node failure, the system must recover within 60 seconds with zero committed data loss.

### 4.4 Usability and Accessibility

- **Cross-Platform:** The mobile application must maintain feature parity across iOS 14+ and Android 9+.
- **Accessibility:** The mentor dashboard must comply with WCAG 2.1 AA standards for contrast and keyboard navigability.

# Chapter 5

## System Architecture

This chapter delineates the end-to-end system architecture of the *Scavenger Hunt* platform. It specifies the architectural paradigms, technology stack selections, functional components, communication protocols, and infrastructure deployment models required to deliver a secure, high-concurrency, real-time treasure hunt experience.

### 5.1 Architecture Overview

The *Scavenger Hunt* platform is engineered following a modular, multi-tier architectural paradigm designed for high availability, low-latency state synchronization, and horizontal scalability. The platform decouples user interactions, business logic execution, and persistent storage into three distinct logical tiers: the Presentation Tier, the Application Tier, and the Data Tier.

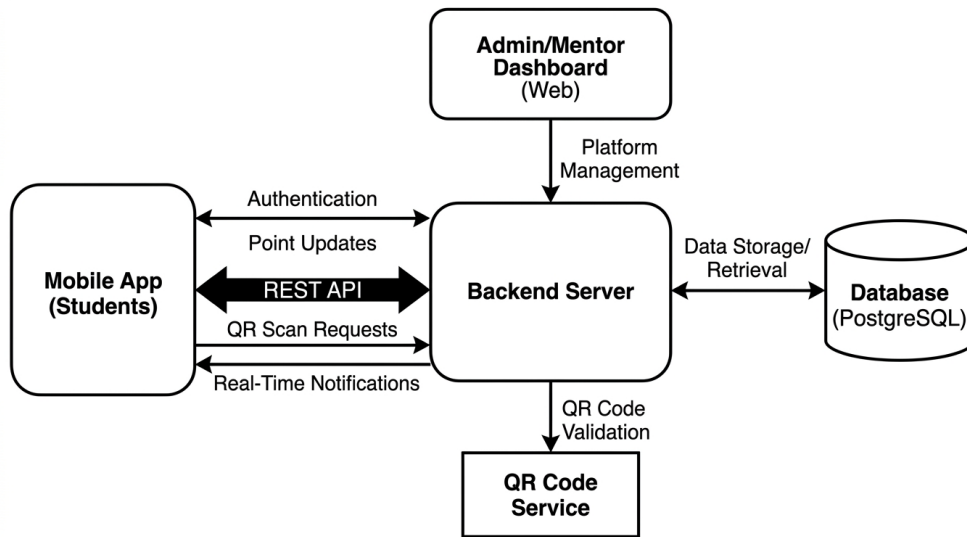


Figure 5.1: System Architecture Overview

As depicted in Figure 5.1, the architecture coordinates client applications, backend application services, real-time messaging subsystems, and transactional databases. The structural responsibilities of each tier are defined as follows:

- **Presentation Tier (Client Layer):** Comprises the cross-platform mobile application utilized by participating student teams and the responsive web dashboard operated by mentors and platform administrators. The mobile client provides the primary interface for team formation, clue consumption, camera-based QR code

scanning, real-time countdown tracking, and leaderboard viewing. The web dashboard provides administrative oversight, allowing mentors to configure hunt stages, monitor active team trajectories, issue manual penalty adjustments, and broadcast platform-wide announcements.

- **Application Tier (Business Logic Layer):** Hosted within a hardened server environment, this layer encapsulates the core business rules of the treasure hunt. It exposes a dual-interface model consisting of a stateless RESTful Application Programming Interface (API) for transactional operations and a persistent bidirectional WebSocket gateway for real-time telemetry. An Nginx reverse proxy serves as the unified entry point, routing client traffic, enforcing SSL/TLS termination, and distributing load across containerized backend service instances.
- **Data Tier (Persistence and Caching Layer):** Combines an ACID-compliant relational database management system (PostgreSQL) for persistent storage of structured entities (e.g., user profiles, teams, clues, game sessions, audit logs) with an in-memory data store (Redis) for ephemeral, high-throughput operations such as real-time leaderboard scoring, session state caching, and pub/sub message distribution.

## 5.2 Technology Stack

The selection of technologies for the *Scavenger Hunt* platform is governed by requirements for cross-platform portability, high I/O throughput, strict transactional integrity, and low-latency real-time synchronization. Table 5.1 summarizes the primary technological components across the platform.

Table 5.1: System Technology Stack Specification

| Layer / Domain        | Technology Selected    | Functional Role and Rationale                                                                                                                    |
|-----------------------|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Frontend Mobile       | React Native / Flutter | Cross-platform native mobile application (iOS and Android) providing access to camera hardware, push notifications, and local secure storage.    |
| Frontend Web (Mentor) | React.js               | Single Page Application (SPA) dashboard for mentors and administrators offering interactive data grids, real-time maps, and configuration forms. |

*Continued on next page*

Table 5.1 – *Continued from previous page*

| Layer / Domain           | Technology Selected     | Functional Role and Rationale                                                                                                                       |
|--------------------------|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Backend Runtime          | Node.js with Express.js | Asynchronous, event-driven runtime delivering high concurrency handling for non-blocking I/O operations and REST endpoint routing.                  |
| Primary Database         | PostgreSQL              | Relational DBMS enforcing relational integrity, transactional ACID guarantees, and complex relational querying for teams, stages, and audit trails. |
| In-Memory Cache          | Redis                   | High-speed in-memory store utilizing Sorted Sets (ZSET) for sub-millisecond leaderboard computation and pub/-sub messaging.                         |
| Authentication / OTP     | Twilio / Firebase Auth  | Managed SMS verification and multi-factor One-Time Password (OTP) delivery service for passwordless participant authentication.                     |
| QR Code Engine           | qrcode / Mobile Scanner | Cryptographic QR code generation library on the backend paired with hardware-accelerated camera scanning on mobile clients.                         |
| Hosting & Infrastructure | AWS / DigitalOcean      | Cloud virtual private server (VPS) infrastructure with container orchestration, managed storage, and object storage for asset hosting.              |

*Continued on next page*

Table 5.1 – *Continued from previous page*

| Layer / Domain             | Technology Selected   | Functional Role and Rationale                                                                                                      |
|----------------------------|-----------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Real-Time Gateway          | WebSocket (Socket.io) | Full-duplex bidirectional communication channel enabling instant leaderboard updates, synchronized timers, and elimination alerts. |
| Reverse Proxy / Web Server | Nginx                 | High-performance HTTP server managing reverse proxying, SSL/TLS certificate termination, load balancing, and static asset caching. |

### 5.2.1 Frontend Mobile Architecture

The mobile application is developed using a unified cross-platform framework (React Native or Flutter) to ensure native runtime performance across iOS and Android devices from a single codebase. Key client-side architectural features include:

- **Hardware Abstraction Layer:** Native device integration for optical camera scanning, vibration feedback on clue submission, and localized push notification processing.
- **Offline-Resilient State Management:** Client-side caching of static clue assets and current stage states using encrypted local storage (e.g., SQLite / AsyncStorage) to mitigate intermittent cellular connectivity.
- **Socket Client Lifecycle Management:** Intelligent reconnection policies with exponential backoff to re-establish WebSocket connections during network handoffs.

### 5.2.2 Frontend Web Dashboard Architecture

The mentor and administration portal is implemented as a single-page web application using React.js. It features a component-based architecture organized around role-based access control (RBAC). The dashboard communicates with the backend via authenticated REST endpoints for configuration tasks and maintains an active WebSocket subscription to ingest live telemetry, team check-ins, and panic alerts.

### 5.2.3 Backend Application Runtime

The backend service utilizes Node.js and Express.js, capitalizing on its single-threaded, non-blocking asynchronous event loop to handle concurrent client connections efficiently. The backend is structured into modular domain controllers, service layers, data access objects (DAOs), and middleware pipelines for authentication, validation, rate limiting, and global error handling.

### 5.2.4 Persistence and Caching Architecture

The storage strategy separates durable relational data from ephemeral, high-speed game state:

- **PostgreSQL:** Manages all persistent relational entities, including **Users**, **Teams**, **Games**, **Stages**, **Clues**, **Submissions**, and **AuditLogs**. Relational foreign key constraints and transactional boundaries prevent state corruption during concurrent submissions.
- **Redis:** Operates as a fast caching layer and specialized data engine. Redis Sorted Sets (**ZSET**) store team scores and completion timestamps, enabling  $O(\log(N) + M)$  retrieval of ranked leaderboards across thousands of active participants. Redis also provides distributed session management and token blacklisting.

## 5.3 Component Diagram and Subsystems

The platform backend is decomposed into specialized functional services, each encapsulating a discrete domain of business logic. Figure 5.1 illustrates the interaction between these subsystems, which are detailed below.

### 5.3.1 Authentication Service

The Authentication Service manages user identity, role provisioning, and session validation.

- **Responsibilities:** Processing mobile participant phone number inputs, interfacing with the SMS provider to dispatch 6-digit numeric OTPs, validating returned tokens, issuing signed JSON Web Tokens (JWTs), and verifying mentor credential pairs.
- **Security Mechanisms:** Short-lived access tokens (15-minute expiration) paired with secure refresh tokens stored in HTTP-only cookies or secure mobile keychains; rate limiting on OTP dispatch endpoints to prevent toll fraud.

### 5.3.2 Game Engine Service

The Game Engine is the core orchestration module governing treasure hunt state transitions.

- **Responsibilities:** Enforcing game progression rules, sequencing clues per team route, validating stage unlock conditions, evaluating countdown timers, computing stage completion scores, and triggering team disqualifications upon timeout or rule violations.
- **State Machine:** Manages strict phase transitions (**PENDING**, **ACTIVE**, **STAGE\_IN\_PROGRESS**, **STAGE\_COMPLETED**, **ELIMINATED**, **FINISHED**) to guarantee deterministic state flow.

### 5.3.3 QR Code and Validation Service

The QR Service oversees the cryptographic generation and verification of physical checkpoint markers.

- **Responsibilities:** Generating time-bound, cryptographically signed QR code payloads for each checkpoint; validating scanned tokens submitted by team mobile clients; ensuring one-time-use constraints per stage; and detecting replay attacks or location spoofing.
- **Payload Security:** QR tokens embed a cryptographically hashed signature comprising the Stage ID, Checkpoint Identifier, Game Session ID, and an HMAC secret salt to prevent unauthorized manual reproduction.

### 5.3.4 Notification and Real-Time Service

The Notification Service manages outward communication to mobile participants and web dashboard operators.

- **Responsibilities:** Pushing immediate state changes over WebSocket channels; broadcasting global countdown updates; distributing stage completion alerts; and pushing critical SMS or in-app notifications regarding game anomalies, weather warnings, or mentor broadcasts.

### 5.3.5 Admin and Mentor Management Service

The Admin Service provides management interfaces for operational staff.

- **Responsibilities:** Creation, editing, and deletion of treasure hunt routes, clue sets, and checkpoint coordinates; live monitoring of team progress across map overlays; manual override capabilities (e.g., granting score adjustments, resetting stuck team states); and generation of final post-game audit analytics and CSV export reports.

## 5.4 Communication Protocols

The *Scavenger Hunt* system relies on a hybrid communication architecture tailored to the operational requirements of each interaction model.

### 5.4.1 RESTful API Specification

Stateless HTTP/HTTPS REST endpoints serve CRUD operations, configuration changes, and non-real-time data queries.

- **Transport Protocol:** HTTPS over TLS 1.3 on TCP port 443.
- **Data Interchange Format:** UTF-8 encoded JSON with standard payload envelopes containing `status`, `data`, `message`, and `timestamp`.
- **Authentication Mechanism:** Authorization headers containing bearer JWTs (`Authorization: Bearer <token>`).

- **Standard HTTP Status Codes:** Consistent use of RFC 7231 status codes (e.g., 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests, 500 Internal Server Error).

### 5.4.2 Real-Time WebSocket Protocol

To achieve instantaneous synchronization across all connected clients without polling overhead, the platform establishes full-duplex WebSocket connections using Socket.io.

- **Transport Protocol:** Secure WebSockets (WSS) upgraded over HTTPS port 443.
- **Connection Lifecycle:** Clients authenticate during the initial handshake by transmitting their JWT in the connection payload. Upon verification, the connection is assigned to specific socket rooms partitioned by `gameId`, `teamId`, and `role`.
- **Event Taxonomy:**
  - `leaderboard:update`: Broadcasts updated top-tier team rankings and point deltas when a checkpoint is verified.
  - `game:timer_sync`: Periodically broadcasts the global authoritative server clock and remaining stage duration.
  - `team:eliminated`: Transmits targeted notifications to disqualified teams and updates the mentor dashboard.
  - `stage:unlocked`: Dispatches the subsequent clue payload directly to the authenticated team socket room.
- **Heartbeat and Keep-Alive:** Automatic ping/pong frames every 25 seconds with a 20-second timeout window to detect and prune dead connections rapidly.

### 5.4.3 SMS Gateway Protocol

Communication with the SMS carrier network is handled asynchronously via HTTPS REST webhooks and REST client SDKs.

- **Transport Protocol:** HTTPS REST with Basic / API Key authentication over TLS 1.3.
- **Delivery Policy:** OTP generation enforces an expiration TTL of 300 seconds (5 minutes) and a rate limit of one OTP request per phone number every 60 seconds to mitigate denial-of-service and SMS toll fraud.

## 5.5 Deployment Architecture

The deployment architecture is engineered for resilience, container isolation, and simplified continuous integration/continuous deployment (CI/CD) pipelines.

### 5.5.1 Containerization and Service Isolation

All backend services, frontend web assets, and background workers are packaged into immutable Docker containers. Multi-stage Dockerfiles are employed to minimize final

image footprints and eliminate build toolchains from runtime production images. A container orchestration layer (Docker Compose for staging/single-node deployments or Kubernetes for clustered deployments) coordinates service lifecycles, health checks, and restart policies.

### 5.5.2 Nginx Reverse Proxy and Traffic Routing

An Nginx reverse proxy sits at the edge of the server infrastructure, serving as the sole public gateway. Key operational responsibilities include:

- **Traffic Routing:** Directing `/api/*` routes to backend application containers, `/socket.io/*` routes to WebSocket handlers with HTTP/1.1 protocol upgrade headers, and root requests to static React.js production build artifacts.
- **Gzip / Brotli Compression:** Compressing JSON responses and static assets on the fly to conserve mobile bandwidth.
- **Rate Limiting:** Enforcing leaky-bucket rate limiting per IP address to safeguard public endpoints against brute-force and DDoS attacks.

### 5.5.3 SSL/TLS Termination and Security

All external traffic is strictly encrypted in transit. SSL/TLS termination is handled at the Nginx edge using automated Let's Encrypt certificates managed via Certbot. The configuration enforces:

- Exclusive support for modern cryptographic protocols (TLS 1.2 and TLS 1.3).
- HTTP Strict Transport Security (HSTS) with standard preloading headers.
- Automatic redirection of all insecure HTTP port 80 requests to encrypted HTTPS port 443.

### 5.5.4 Data Persistence and Backup Strategy

The deployment maintains volume mount isolation for persistent data stores:

- **PostgreSQL Volume Persistence:** Dedicated Docker volume storage with scheduled automated `pg_dump` snapshots stored in off-site S3-compatible object storage.
- **Redis Persistence Configuration:** Configured with Append-Only File (AOF) logging combined with periodic RDB snapshots to guarantee in-memory state recovery across container restarts.

# Chapter 6

## Database Design

### 6.1 Overview

The *Scavenger Hunt* platform requires a robust, relational data architecture capable of supporting real-time gamification and concurrent QR code verification. The underlying persistence layer is built on **PostgreSQL 16**, chosen for its enterprise-grade transactional guarantees (ACID compliance).

### 6.2 Entity-Relationship Model

The domain model captures the complex interactions between human operators (Users, Players, Mentors), game state constructs (Games, Rounds), physical locations (Indexes), and verified gameplay events (Scans).

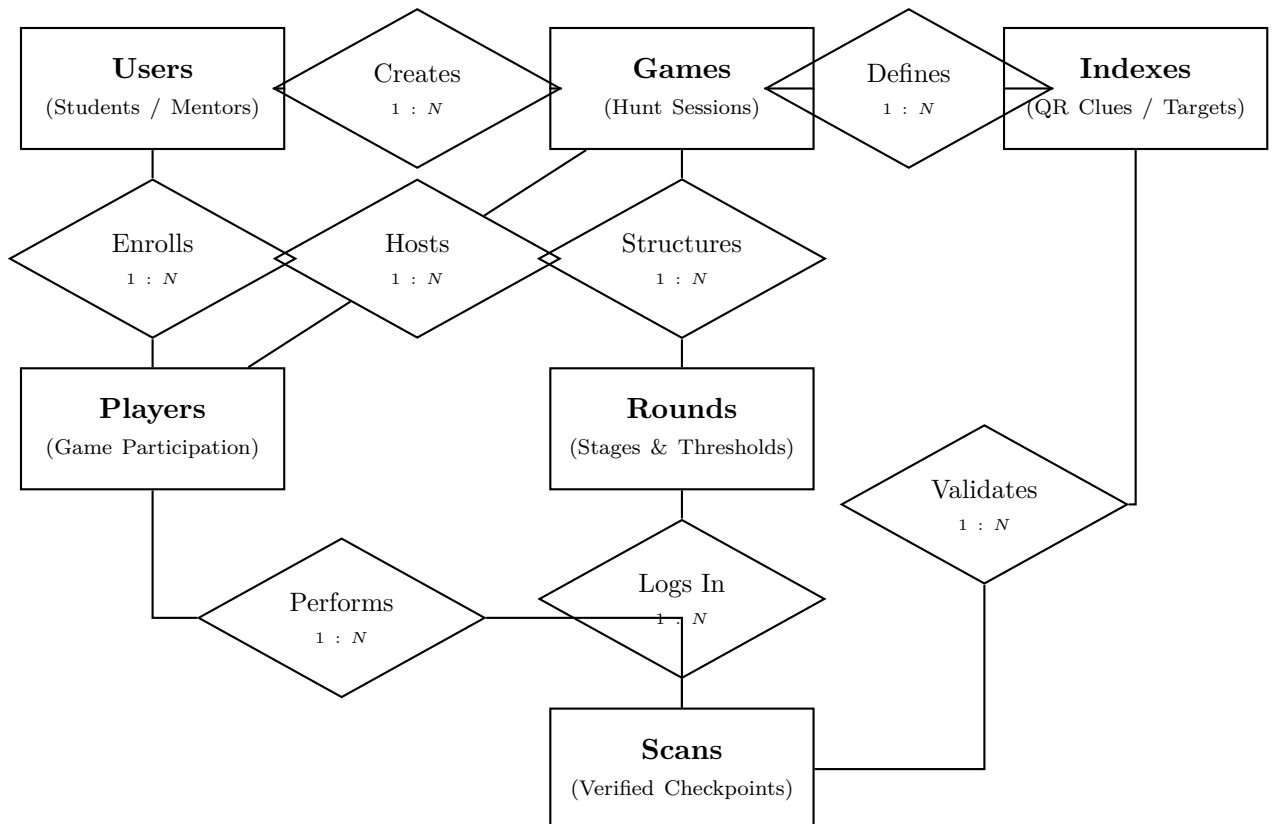


Figure 6.1: Entity-Relationship Overview of the Scavenger Hunt Data Model

## 6.3 Core Tables Summary

- **Users:** Stores profile information (phone number, nickname, role).
- **Games:** Defines the root configuration for a scavenger hunt instance.
- **Rounds:** Sequential phases of a game with defined score thresholds and durations.
- **Indexes:** Geographic checkpoints and cryptic clues.
- **Players:** Joins users to specific game instances, tracking total score and current round status.
- **Scans:** Immutable audit log of every successful QR code validation.

# Chapter 7

## User Interfaces and UX Architecture

### 7.1 Overview

The UI/UX architecture of the *Scavenger Hunt* platform bridges two distinct operational environments: high-tempo, physical outdoor exploration for student participants (Mobile App), and centralized, real-time event telemetry for mentors (Web Dashboard).

### 7.2 Student Mobile Application (Frontend)

The mobile application is a cross-platform (iOS/Android) interface designed for rapid interaction under outdoor lighting conditions.

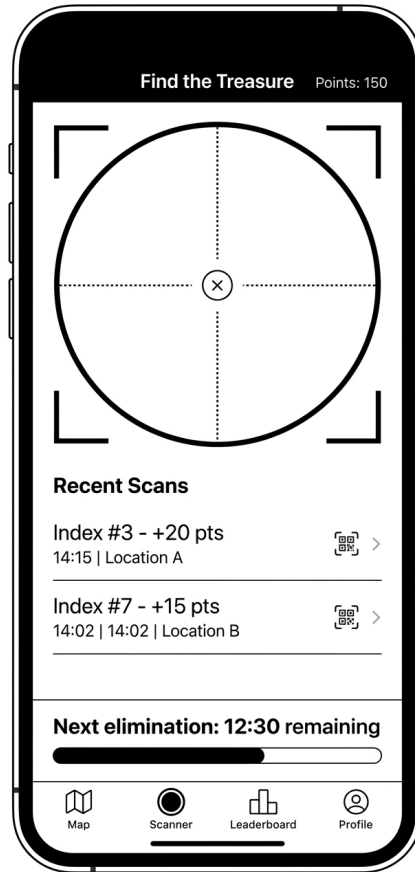


Figure 7.1: Conceptual Wireframe of the Student Mobile Interface

### 7.2.1 Core Mobile Screens

- **Authentication & Lobby:** SMS verification entry and a waiting lobby displaying the countdown to the game launch.
- **Active Stage Dashboard:** Displays the current cryptic clue, a prominent ‘Scan QR’ action button, and a visual countdown timer for the current round.
- **Camera Scanner Overlay:** Full-screen viewfinder with edge detection reticles for rapid QR acquisition.
- **Live Leaderboard:** A real-time sliding list of competing teams, explicitly highlighting the elimination threshold line (e.g., the bottom 20%).

## 7.3 Mentor Web Dashboard (Backend Admin)

The mentor portal is a centralized Single Page Application (SPA) optimized for desktop and tablet resolutions.

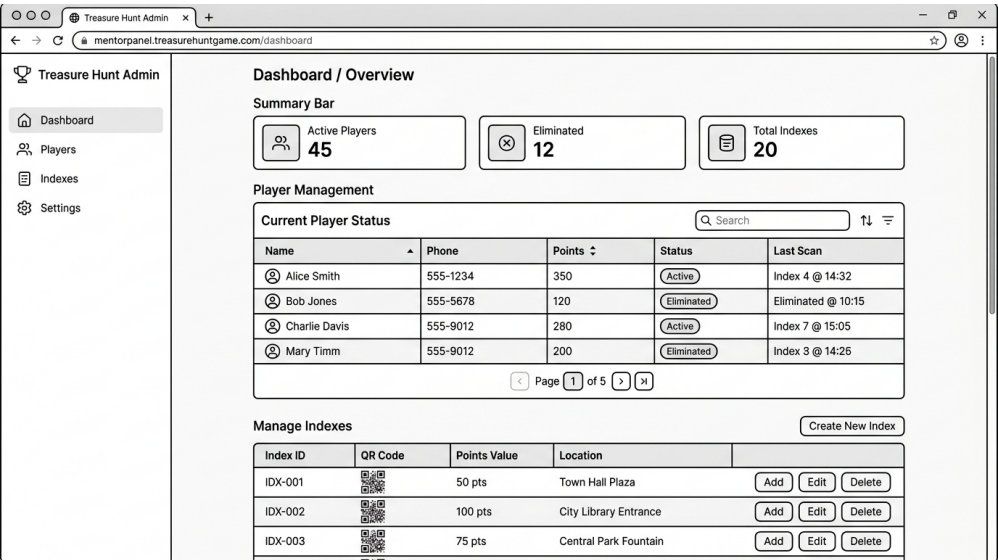


Figure 7.2: Conceptual Layout of the Mentor Web Dashboard

### 7.3.1 Core Dashboard Modules

- **Game Control Center:** Master controls to instantiate games, trigger stage transitions, or halt gameplay globally.
- **Live Telemetry Map:** A geographical overlay plotting real-time team coordinates and scanning velocities.
- **Audit Log & Disqualification Matrix:** A chronologically ordered data grid tracking every scan event, allowing mentors to manually disqualify teams for rules infractions.

# Chapter 8

## Game Mechanics and Business Rules

### 8.1 Overview

This chapter specifies the core game mechanics and algorithmic lifecycle governing the *Scavenger Hunt* platform. The platform orchestrates multi-round, campus-wide physical scavenger hunts where participants locate physical QR codes, decode clues, and survive elimination rounds.

### 8.2 State Machine Lifecycle

The lifecycle of a *Scavenger Hunt* game instance is managed as a deterministic Finite State Machine (FSM):

1. **PENDING:** The game is scheduled. Players join teams using invite codes.
2. **ACTIVE:** The game is live. The timer for the first round begins.
3. **ELIMINATING:** The round timer has expired. The system halts scanning, ranks teams by score, and eliminates the lowest percentile.
4. **FINISHED:** The final round completes, declaring the winning team.

### 8.3 Scoring and Progression

- **Base Points:** Each valid QR scan awards a fixed amount of base points.
- **Time Multiplier:** Scanning an index earlier in the round yields a higher time-based bonus.
- **Dynamic Routing:** Teams may receive randomized index assignments to prevent swarming at a single physical location.

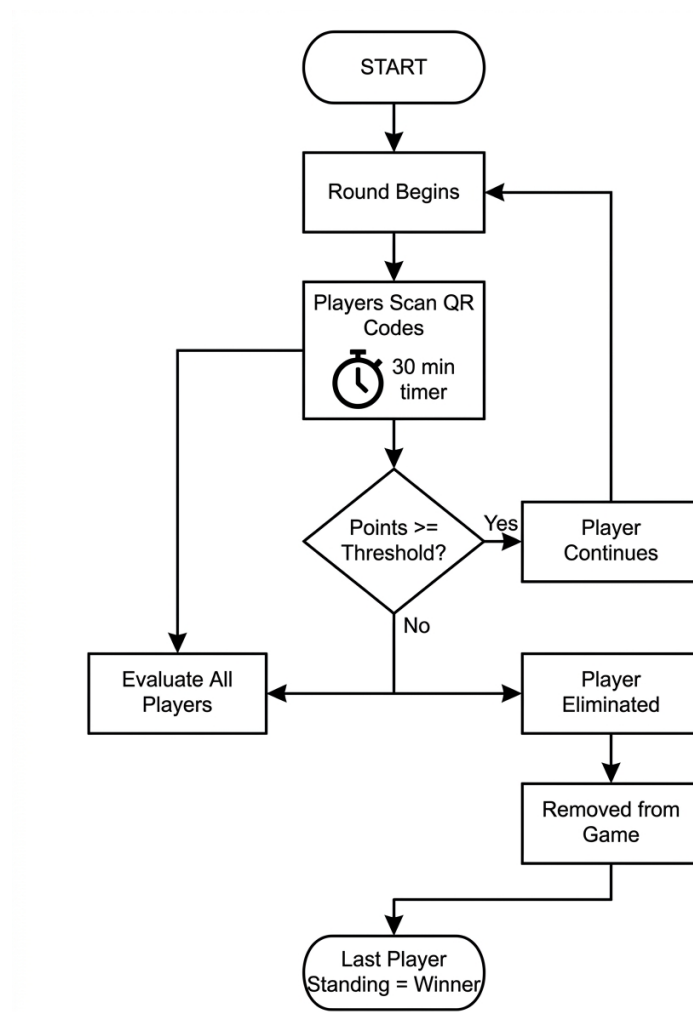


Figure 8.1: Logical Game Flow and State Progression Diagram

# Chapter 9

## API Specification

### 9.1 API Overview

The ‘Scavenger Hunt’ platform exposes a comprehensive, RESTful Application Programming Interface (API) complemented by real-time WebSocket communication channels. The API serves as the primary communication bridge between the client applications and the backend services.

#### 9.1.1 Architectural Principles

The platform API adheres strictly to REST design principles:

- **Stateless Interactions:** No client session state is retained on the server.
- **Resource-Oriented URI Design:** Endpoints are structured hierarchically.
- **JSON Payload Representation:** All payloads utilize the `application/json` MIME type.

#### 9.1.2 Authentication and Rate Limiting

All non-public API routes require an authenticated identity supplied via JSON Web Tokens (JWT) in the `Authorization` header. Endpoints are protected by sliding-window rate limiters to prevent abuse.

### 9.2 Core Endpoints Summary

The API is divided into logical domains. Table 9.1 provides a high-level summary of the primary REST resources.

### 9.3 WebSocket Events

Real-time state synchronization is handled via WebSocket channels.

- **Server-to-Client:** `game:state_changed`, `team:eliminated`, `leaderboard:updated`.
- **Client-to-Server:** `team:ping` (location check-in), `player:alert` (SOS/Help request).

Table 9.1: API Endpoints Summary

| Domain | Endpoint / Resource                 | Description                                         |
|--------|-------------------------------------|-----------------------------------------------------|
| Auth   | POST /api/v1/auth/send-otp          | Dispatches a passwordless OTP via SMS.              |
|        | POST /api/v1/auth/verify-otp        | Verifies OTP and returns JWT tokens.                |
| Player | GET /api/v1/players/me              | Retrieves current player profile.                   |
|        | PUT /api/v1/players/me              | Updates player nickname and avatar.                 |
|        | POST /api/v1/players/me/team        | Joins or creates a team using an invite code.       |
| Game   | GET /api/v1/games/active            | Retrieves the currently active game session.        |
|        | GET /api/v1/games/{id}/leaderboard  | Fetches the live leaderboard for a specific game.   |
|        | POST /api/v1/games/{id}/scan        | Submits a QR code scan for validation.              |
| Mentor | POST /api/v1/admin/games            | Provisions a new scavenger hunt instance.           |
|        | POST /api/v1/admin/games/{id}/state | Progresses the game state (e.g., START, ELIMINATE). |
|        | GET /api/v1/admin/teams             | Retrieves detailed telemetry for all teams.         |

# Chapter 10

## Security Architecture

### 10.1 Security Overview

The “Scavenger Hunt” platform coordinates mobile players, physical QR code waypoints, mentor dashboards, and an authoritative backend. The security architecture assumes a *zero-trust* environment where any client device or communication link could be compromised.

### 10.2 Core Security Controls

#### 10.2.1 Authentication and Authorization

- **Passwordless SMS OTP:** Mitigates brute-force credential stuffing and password reuse by relying on out-of-band SMS verification.
- **JWT Bearer Tokens:** Short-lived access tokens (15-minute expiry) and rotating refresh tokens stored securely in HTTP-only cookies or native mobile keychains.
- **Strict RBAC:** Explicit isolation between `ROLE_PLAYER` and `ROLE_MENTOR` permissions at the API routing layer.

#### 10.2.2 Anti-Cheat and Validation

- **Cryptographic QR Codes:** QR payloads embed HMAC signatures incorporating the Stage ID and Game Session ID to prevent manual reproduction or guessing of endpoint URLs.
- **Rate Limiting:** Sliding-window limits prevent automated scanning scripts or API abuse.
- **Geolocation Auditing:** Optional client-side GPS coordinates appended to submissions for mentors to audit implausible movement speeds between checkpoints.

#### 10.2.3 Infrastructure Security

- **TLS Encryption:** All API (HTTPS) and WebSocket (WSS) traffic is encrypted in transit using TLS 1.3 to prevent man-in-the-middle packet sniffing.
- **Database Isolation:** Parameterized SQL queries via an ORM protect against SQL injection. Row-level security restricts data access across concurrent game instances.

# Chapter 11

## Testing and Quality Assurance

### 11.1 Overview

Rigorous quality assurance is vital for the *Scavenger Hunt* platform due to its high-concurrency, time-sensitive nature, and physical-digital interactions. This chapter outlines the testing strategy.

### 11.2 Testing Methodologies

- **Unit Testing:** Backend services (e.g., scoring logic, QR validation) are isolated and tested using Jest.
- **Integration Testing:** Verification of database transactions and WebSocket pub/sub propagation using Supertest.
- **End-to-End (E2E) Testing:** Automated UI testing of the React Native client and Angular dashboard using Cypress and Appium.
- **Load and Stress Testing:** Artillery is used to simulate thousands of concurrent QR scans at the exact moment a round ends, verifying Redis and PostgreSQL throughput.

### 11.3 Field Testing Protocols

Prior to deployment, the platform undergoes a physical "Dry Run" on campus to verify GPS accuracy, cellular dead zones, and optical scanner performance under direct sunlight.

# Chapter 12

## Deployment and Operations

### 12.1 Overview

The operational success of the *Scavenger Hunt* platform depends on a resilient, scalable, and automated deployment infrastructure. This chapter details the cloud infrastructure and CI/CD pipelines.

### 12.2 Infrastructure as Code

The platform's infrastructure is provisioned using Terraform, ensuring reproducible deployments across staging and production environments.

- **Container Orchestration:** Docker containers for the Node.js backend are managed via Kubernetes (or AWS ECS) for automatic scaling.
- **Managed Database:** A managed PostgreSQL RDS instance provides automated daily backups, point-in-time recovery, and read replicas.
- **Cache Node:** A dedicated ElastiCache Redis cluster handles volatile state and pub/sub.

### 12.3 Continuous Integration and Deployment (CI/CD)

GitHub Actions orchestrates the build pipeline:

1. Code pushed to main triggers automated linting (ESLint) and unit tests.
2. Successful builds generate Docker images, pushing them to the Elastic Container Registry (ECR).
3. The staging environment automatically pulls the latest image for QA review before manual promotion to production.

### 12.4 Observability

Prometheus and Grafana provide real-time infrastructure monitoring, while Sentry captures unhandled application exceptions.

# Appendix A

## Appendix

### A.1 QR Code Sample Format

Each QR code generated by the platform encodes a JSON payload, Base64-encoded and signed with HMAC-SHA256. The following is a sample decoded payload:

```
1 {  
2   "index_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",  
3   "game_id": "f9e8d7c6-b5a4-3210-fedc-ba0987654321",  
4   "timestamp": "2026-08-25T14:00:00Z",  
5   "signature": "c2VjdXJlX2htYWVfc2lnbmF0dXJlX2hlcmU="
```

Listing A.1: Sample QR Code Payload (Decoded)

The signature is computed as:

$$\text{signature} = \text{HMAC-SHA256}(\text{secret\_key}, \text{index\_id}||\text{game\_id}||\text{timestamp}) \quad (\text{A.1})$$

### A.2 Sample API Request and Response

#### A.2.1 Send OTP Request

```
1 POST /api/v1/auth/send-otp HTTP/1.1  
2 Host: api.findthetreasure.ma  
3 Content-Type: application/json  
4  
5 {  
6   "phone_number": "+212612345678"  
7 }
```

Listing A.2: Send OTP – HTTP Request

```
1 {  
2   "success": true,  
3   "message": "OTP sent successfully",  
4   "data": {  
5     "expires_in": 300,  
6     "phone_number": "+212612345678"  
7   }
```

```
8 }
```

Listing A.3: Send OTP – HTTP Response (200 OK)

### A.2.2 QR Scan Submission

```
1 POST /api/v1/scan HTTP/1.1
2 Host: api.findthetreasure.ma
3 Content-Type: application/json
4 Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...
5
6 {
7   "qr_data": "eyJpbmRleF9pZCI6ImExYjJjM2Q0Li4uIiwia2FtZV9pZCI6
8   ...",
9   "game_id": "f9e8d7c6-b5a4-3210-fedc-ba0987654321"
}
```

Listing A.4: QR Scan – HTTP Request

```
1 {
2   "success": true,
3   "message": "Index scanned successfully",
4   "data": {
5     "index_label": "Index #7 - Library Entrance",
6     "points_earned": 25,
7     "total_points": 150,
8     "rank": 3,
9     "scanned_at": "2026-08-25T14:23:15Z"
10  }
11 }
```

Listing A.5: QR Scan – HTTP Response (200 OK)

## A.3 Environment Variables Reference

Table A.1: Environment Variables

| Variable          | Type    | Description                                     |
|-------------------|---------|-------------------------------------------------|
| DATABASE_URL      | String  | PostgreSQL connection string                    |
| REDIS_URL         | String  | Redis connection string for caching and pub/sub |
| JWT_SECRET        | String  | Secret key for JWT token signing                |
| JWT_EXPIRY        | Integer | Access token expiry in seconds (default: 3600)  |
| HMAC_SECRET       | String  | Secret key for QR code HMAC signatures          |
| TWILIO_SID        | String  | Twilio Account SID for SMS OTP                  |
| TWILIO_AUTH_TOKEN | String  | Twilio Auth Token                               |
| TWILIO_PHONE      | String  | Twilio sender phone number                      |
| PORT              | Integer | Server port (default: 3000)                     |
| NODE_ENV          | String  | Environment: development, staging, production   |
| CORS_ORIGINS      | String  | Comma-separated allowed CORS origins            |

## A.4 Acronyms and Abbreviations

Table A.2: Acronyms and Abbreviations

| Acronym | Definition                                   |
|---------|----------------------------------------------|
| API     | Application Programming Interface            |
| CORS    | Cross-Origin Resource Sharing                |
| CRUD    | Create, Read, Update, Delete                 |
| CSS     | Cascading Style Sheets                       |
| ENSAM   | École Nationale Supérieure d'Arts et Métiers |
| HMAC    | Hash-based Message Authentication Code       |
| HTTP    | Hypertext Transfer Protocol                  |
| HTTPS   | HTTP Secure                                  |
| JSON    | JavaScript Object Notation                   |
| JWT     | JSON Web Token                               |
| OTP     | One-Time Password                            |
| QR      | Quick Response (code)                        |
| REST    | Representational State Transfer              |
| SHA     | Secure Hash Algorithm                        |
| SMS     | Short Message Service                        |
| SQL     | Structured Query Language                    |
| SSL     | Secure Sockets Layer                         |
| TLS     | Transport Layer Security                     |
| UI      | User Interface                               |
| URL     | Uniform Resource Locator                     |
| UUID    | Universally Unique Identifier                |
| UX      | User Experience                              |
| WCAG    | Web Content Accessibility Guidelines         |
| WSS     | WebSocket Secure                             |
| XSS     | Cross-Site Scripting                         |